If the unique rule on the title attribute fails, the max rule will not be checked. Rules will be validated in the order they are assigned.

If the validation fails, the user will be redirected to the previous location and display an error message to the user. If you want to validate a form, you can simply add in a `request` function on your controller which will first validate the incoming parameters before executing the controller. You can also check if the user has the appropriate rights to access a page or update a resource like uploading a file or leaving a comment etc. with the `authorize` method. If the authorize method returns false, a HTTP response with a 403 status code will automatically be returned and your controller method will not execute.

After calling the `errors` method on a `Validator` instance, you will receive an `Illuminate\Support\MessageBag` instance, which has a variety of convenient methods for working with error messages. You can either display the first error message for a field with the `first` method or all the errors with the get method.

```
$errors = $validator->errors();
echo $errors->first('email');
```

OR

```
foreach ($errors->get('email') as $message) {
    // }
```

To retrieve an array of all messages for all fields, use the all method:

```
foreach ($errors->all() as $message) {
    // }
```

Laravel provides defined error messages but you can override them with your own customized messages with the `messages` method. You may pass the custom messages as the third argument to the `Validator::make` method:

```
$messages = [
    'required' => 'The :attribute field is required.',
];
$validator = Validator::make($input, $rules, $messages);
```

In this example, the `:attribute` place-holder will be replaced by the actual name of the field under validation. You may also utilize other place-holders in validation messages. For example:

```
$messages = [
   'same'    => 'The :attribute and :other must match.',
   'size'    => 'The :attribute must be exactly :size.',
```